OPERATING SYSTEM

UNIT 3 ADDRESS SPACE

Prof. N. K. PATIL
Department of Computer Engineering
Sinhgad Institute of Technology, Lonavala

Email nkp.sit@sinhgad.edu
Contact No.+91-9423340927

sppu_te_be_comp

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES
WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

Insta Page

(for all Engg & tech updates)

@SPPU_ENGINEERING_UPDATE

https://www.instagram.com/sppu_engineering_update

sppu_te_be_comp

CONTENTS

1. Early Systems, Multiprogramming and Time Sharing, The address space
2. Virtualization of memory, Memory API: types of memory, the malloc() call, the free() call
3. Segmentation, Fine-grained vs. coarse-grained segmentation
4. Free Space Management, Paging, A memory Trace
5. Faster translations (TLBs),
6. TLB basic algorithm, TLB issue: context switch
7. Replacement Policy, Hybrid approach: paging and segments
8. Beyond Physical Memory: mechanisms, swap space
9. The page fault, page fault control flow.

sppu_te_be_comp

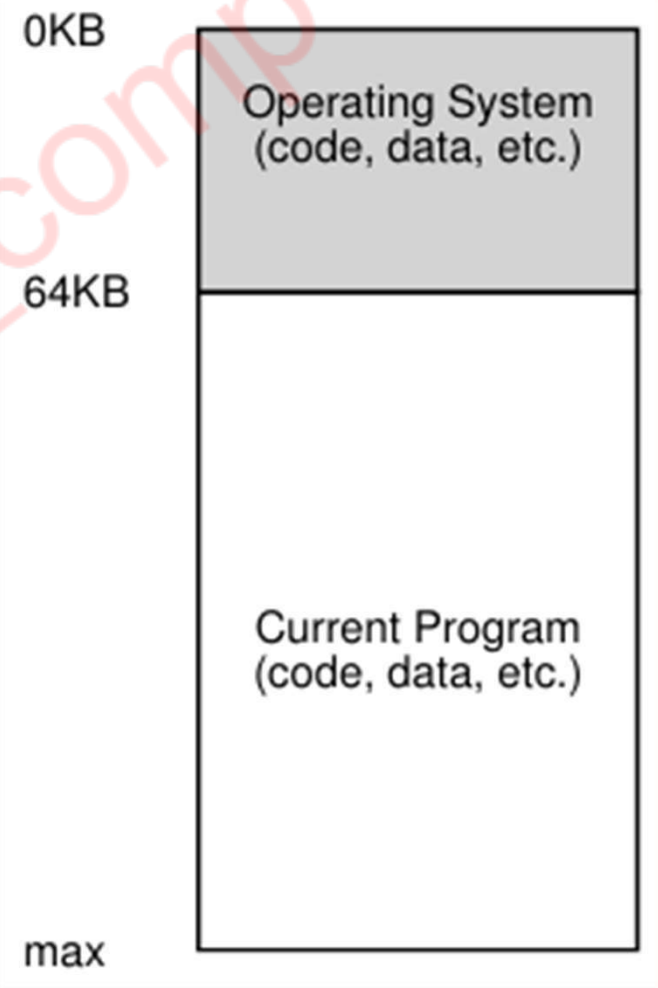
COURSE OUTCOME

- CO3: **Discuss** various memory management techniques.

sppu_te_be_comp

EARLY SYSTEMS

- From the perspective of memory, early machines didn't provide much of an abstraction to users. Basically, the physical memory of the machine looked as shown in fig. below:



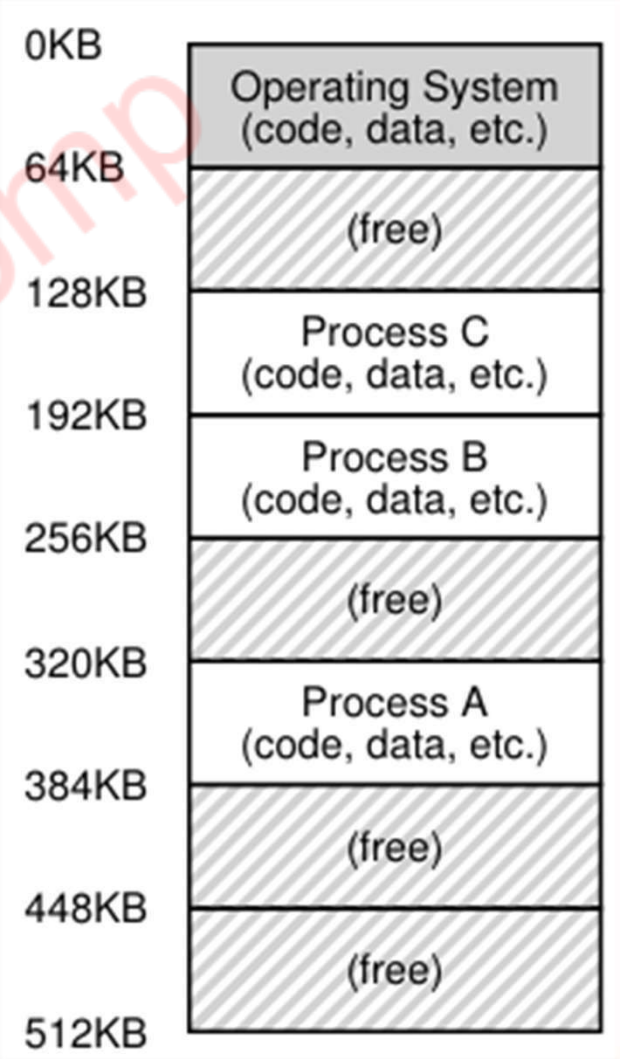
sppu_to_be_comp

MULTIPROGRAMMING & TIME SHARING

- **In multiprogramming, multiple processes were ready to run at a given time, and the OS would switch between them. Doing so increased the effective utilization of the CPU.**
- **After then came the era of time sharing. One way to implement time sharing would be to run one process for a short while, giving it full access to all memory.**

spu_fe_be_comp

- three processes (A, B, and C) and each of them have a small part of the 512-KB physical memory carved out for them.
- Assuming a single CPU, the OS chooses to run one of the processes (say A), while the others (B and C) sit in the ready queue waiting to run.



WHAT DO YOU MEAN BY ADDRESS SPACE IN OS

- **In OS, an address space is a range of memory addresses that a program can use to store and retrieve data, with each program having its own unique, private address space.**
- **This concept involves two types of addresses:**
  - **physical address space,** the actual memory addresses in the computer's hardware, and
  - **virtual address space,** an abstraction that the OS provides to programs, creating the illusion of a large, contiguous memory

spot_to_be_comp

- A logical address is a virtual address used by a program to reference memory locations. It's generated by the CPU during program execution.
- A physical address is the actual hardware-defined location in the computer's main memory (RAM) where data is stored.
- The Memory Management Unit (MMU) which is a hardware component that translates virtual addresses (generated by the CPU) into physical addresses (used by the memory).

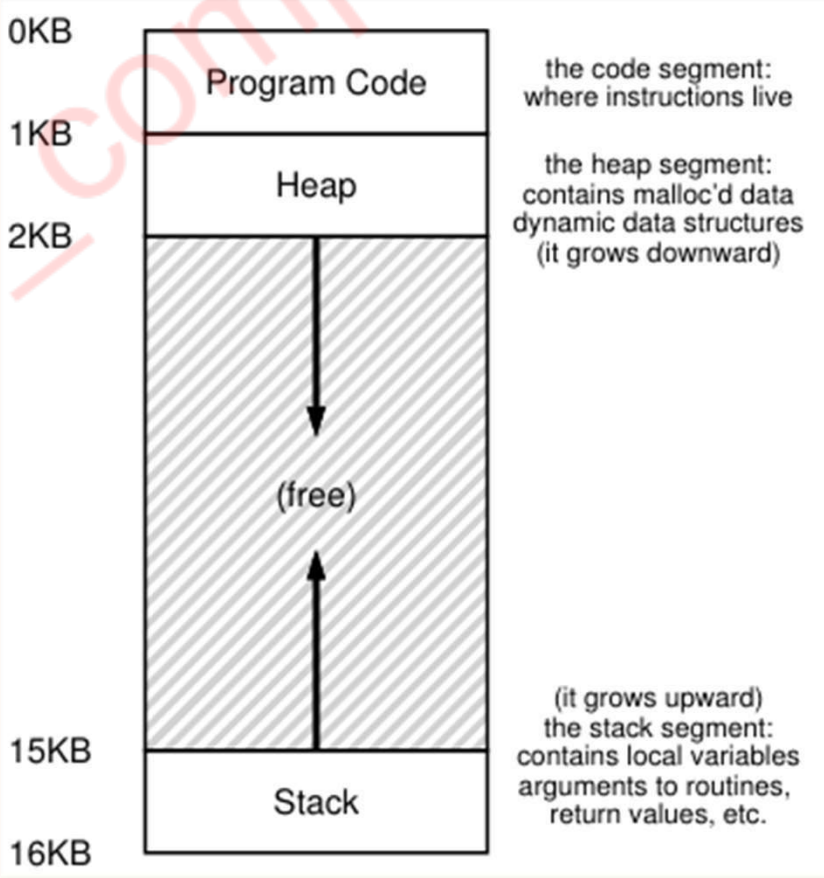
spou_to_be_comp

- **Example-**
  - **Logical Address:** Like a room number on your hotel booking—it's what you see and use, assigned by the system.
  - **Physical Address:** The actual location of that room in the building—known only to the hotel staff (like the OS and MMU).

sppu_te_be_comp

EXAMPLE ADDRESS SPACE

- The address space of a process contains all of the memory state of the running program.
- As seen in the above example program code is static, while two regions of the address space that may grow (and shrink) while the program runs.



- **Modern operating systems use memory virtualization to give each running program the illusion that it has access to a large, private memory space—starting at address 0.**

- **In reality, the program's memory is mapped to different physical locations by the OS, with help from hardware (like the Memory Management Unit).**

sppu_te_be_comp

MEMORY VIRTUALIZATION

- **Memory virtualization in an OS creates the illusion of more memory than physically available by mapping logical addresses to physical memory and using disk storage to extend RAM.**

- **Memory virtualization is a technique used by operating systems to abstract physical memory and present each process with its own private, contiguous address space, regardless of the actual physical memory layout.**

sppu_te_be_comp

- **How Virtual Memory Works**
  - Virtual memory uses both hardware and software to manage memory.
  - When a program runs, it uses virtual addresses (not real memory locations).
  - The computer system converts these virtual addresses into physical addresses (actual locations in RAM) while the program runs.

spput_te_be_comp

- **Example of Virtual memory**
  - Imagine a hotel with many rooms (physical memory). Each guest (process) believes they have the entire floor to themselves (virtual memory). The concierge (OS + MMU) ensures guests are directed to the correct rooms without ever seeing each other.
  - Think of virtual addresses like seat numbers on a boarding pass. Whether you're in the lounge (RAM) or temporarily moved to a waiting area (disk), your seat number doesn't change—but your actual location does.

spru_te_be_comp

TYPES OF MEMORY

- **2 types of memory are allocated to a running program:**
  - **Stack memory:** allocations and deallocations of it are managed implicitly by the compiler for you, it is also known as automatic memory.

```
void func() {
        int x;    // declares an integer on the stack
        ...
}
```

  - **Heap memory:** allocations and deallocations are explicitly handled by the programmer,

```
void func() {
        int *x = (int *) malloc(sizeof(int));      // declares an integer on the heap
        ...
        }
```

sppu_te_be_comp

THE MALLOC() CALL

- malloc() stands for memory allocation.
- It's a function in C used to dynamically allocate memory on the heap during runtime.
- The memory allocated by malloc() is uninitialized, meaning it contains garbage values.
    - malloc(size);
- where size is the number of bytes to allocate.
- This function returns a void pointer to the allocated memory that needs to be converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

sppu_te_be_comp

THE FREE() CALL

- **The memory allocated using function malloc() is not de-allocated on their own.**
- **The free() function is used to release dynamically allocated memory back to the OS.**
- **To free heap memory that is no longer in use, programmers simply call free():**

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

sputte_be_comp

MEMORY MANAGEMENT

- **Main memory is divided into two parts:**
  - One part for the OS (resident monitor, kernel).
  - One part for the program currently being executed.

- **Subdividing memory to accommodate multiple processes.**
- **Memory needs to be allocated efficiently to pack as many processes into memory as possible**

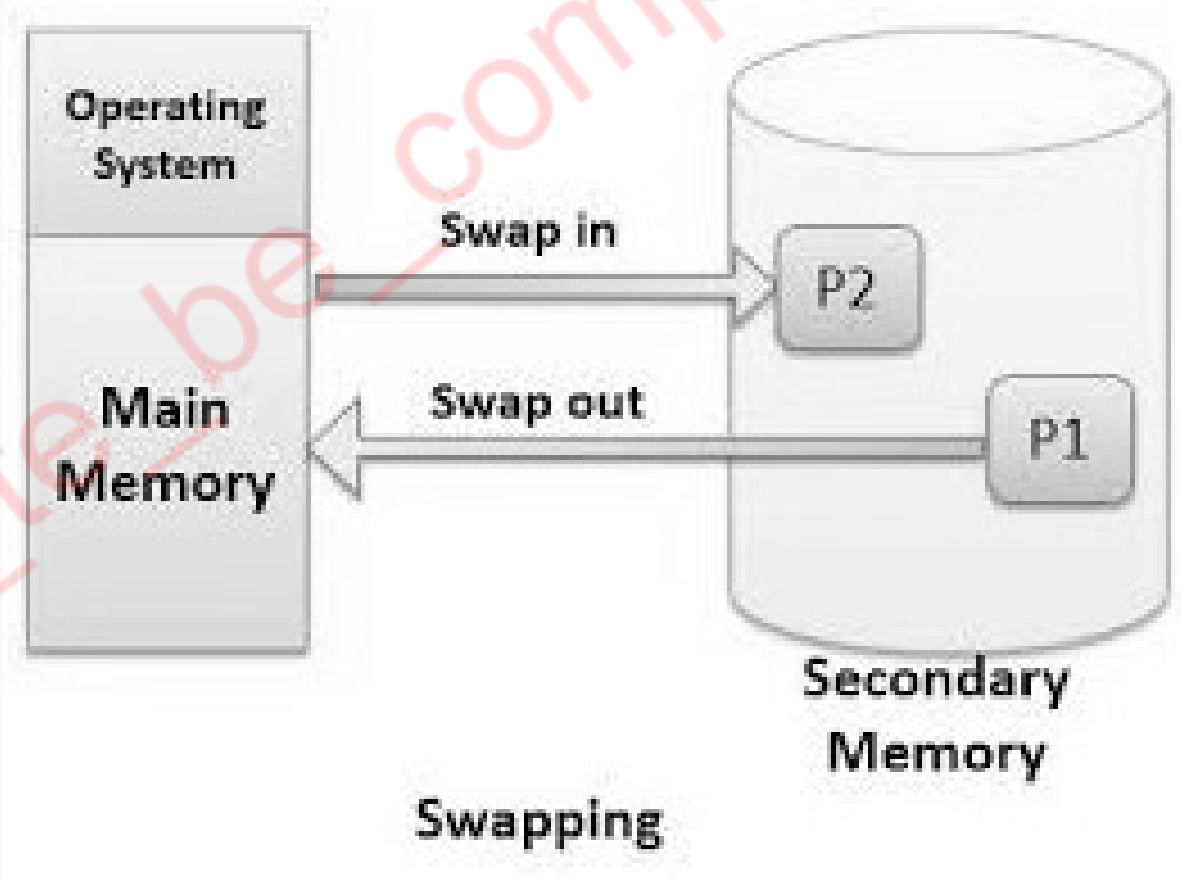
sppu_te_be_comp

- **Memory management terms:**
  - **Frame:** a fixed length block of main memory.
  - **Page:** a fixed length block of data that resides in secondary memory. A page of data may temporarily be copied into a frame of main memory.
  - **Segment:** a variable length block of data that resides in main memory. An entire segment may temporarily be copied into an available region of main memory.

sppu_te_be_comp

SWAPPING

- A process must be in memory to be executed. A process, however, can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.



Swapping

sppu_te_be_comp

MEMORY MANAGEMENT REQUIREMENTS

- **Memory management keeps track of the status of each memory location, whether it is allocated or free.**
- **MM requirements include the following:**
  1. **Relocation**
  2. **Protection**
  3. **Sharing**
  4. **Logical organization**
  5. **Physical organization**

spou_te_be_comp

MEMORY PARTITIONING

- **Memory partitioning in operating systems is a technique used to divide the main memory into sections, or partitions, to allocate memory to different processes.**
- **This allows multiple programs to run concurrently by isolating their memory spaces, preventing them from interfering with each other.**
- **Types of Memory Partitioning:**
  - **Fixed Partitioning**
  - **Dynamic Partitioning**

sppu_te_be_comp

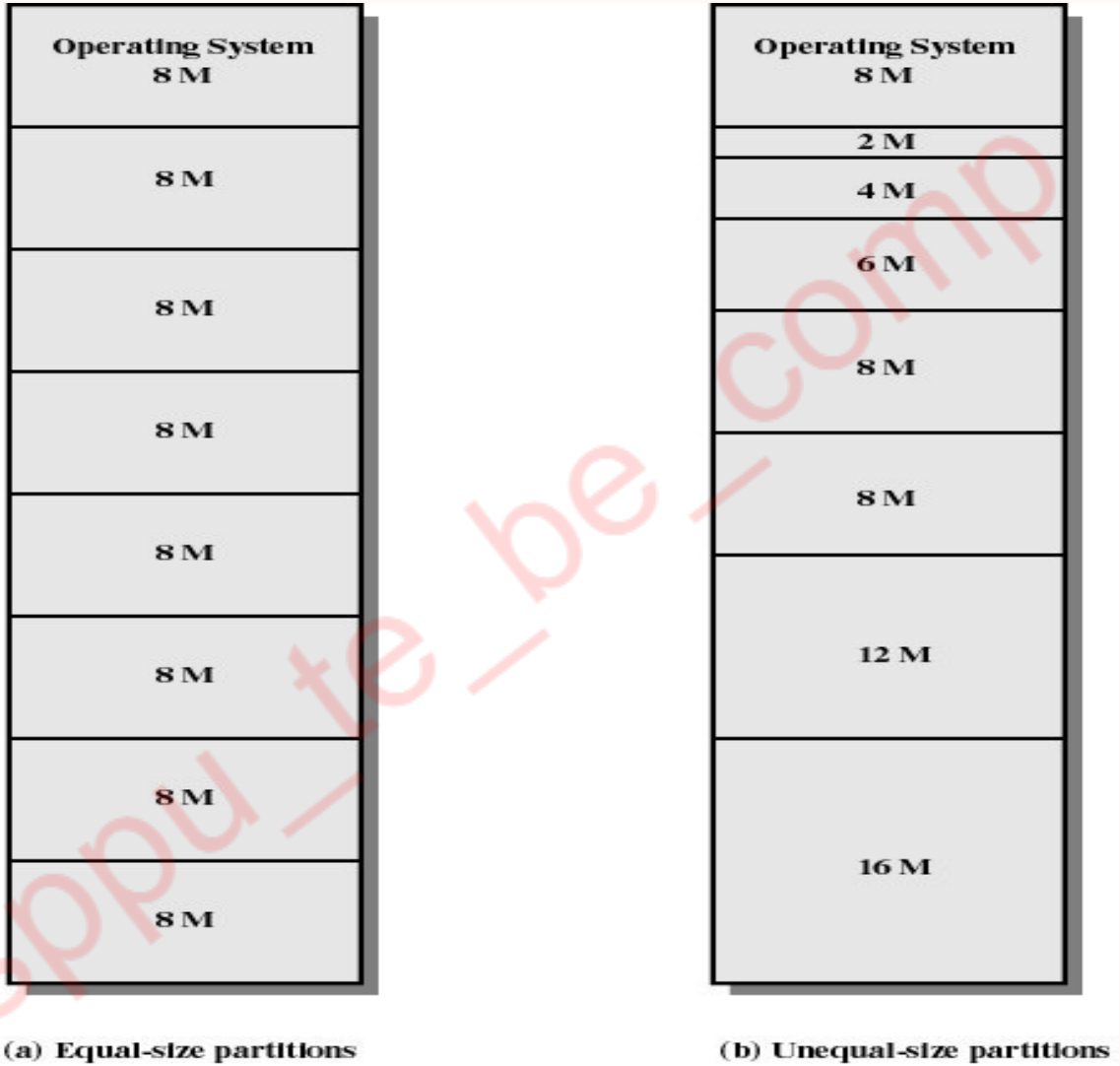
FIXED PARTITIONING

- **The simplest scheme for managing main memory is to partition it into regions with fixed boundaries.**
- **Fixed partitions can be of**
  - **equal or**
  - **unequal sizes.**

sppu_te_be_comp

- **Equal sized Partition –**
  - Any process whose size is less than or equal to the partition size can be loaded into an available partition
  - If all partitions are full, the operating system can swap a process out of any of the partitions and load in another process.

sppu_te_be_comp



sppu_te_be_comp

Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory

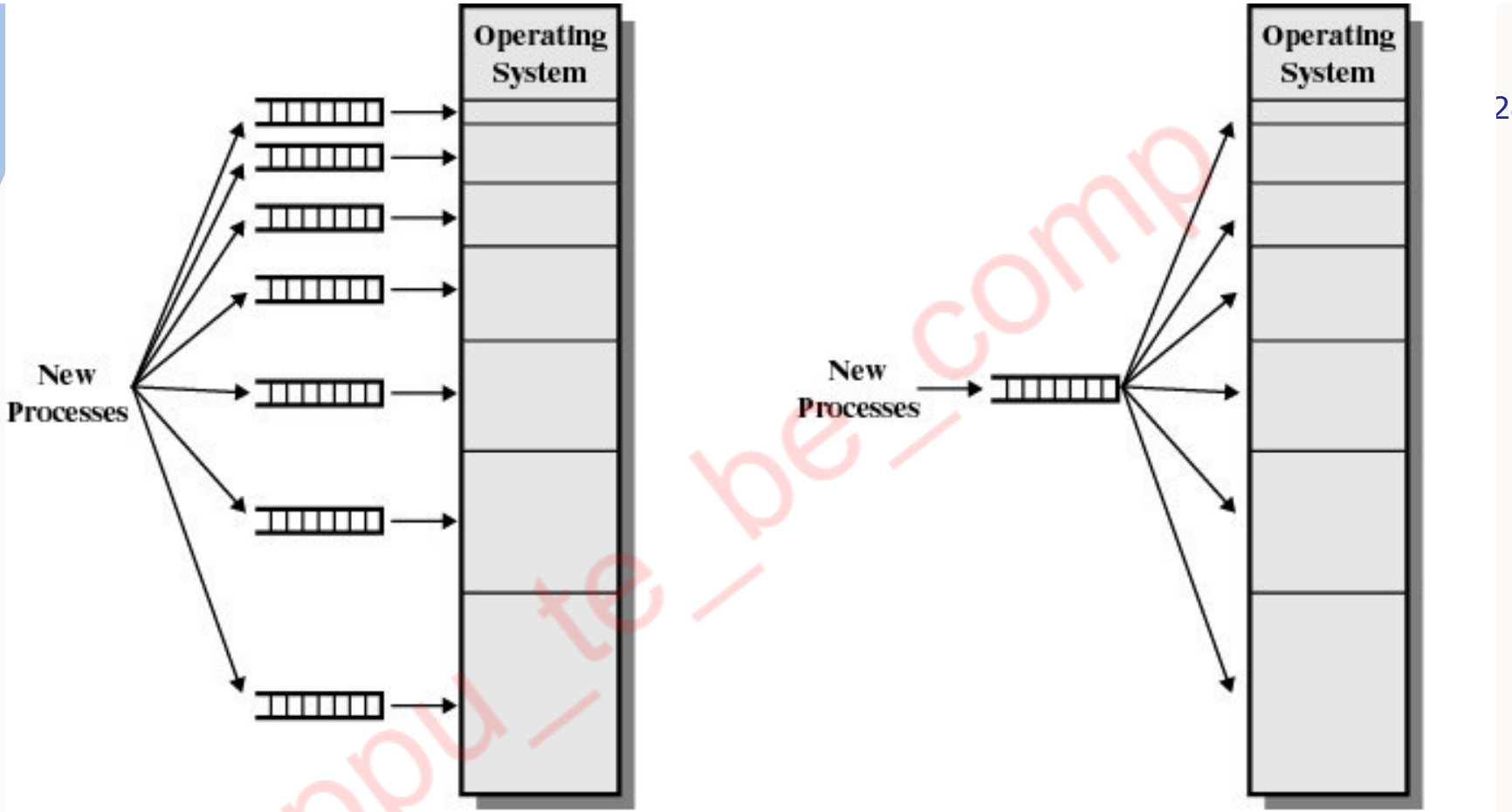
- There are two difficulties with the use of equal-size fixed partitions:
  - A program may be too big to fit into a partition. The programmer must design the program fix this with the concept of **overlays**.
  - Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This is called **internal fragmentation**.

spu_te_be_comp

PLACEMENT ALGORITHM WITH PARTITIONS

- **Equal-size partitions**
  - **because all partitions are of equal size, it does not matter which partition is used**
- **Unequal-size partitions**
  - **can assign each process to the smallest partition within which it will fit**
  - **queue for each partition**
  - **processes are assigned in such a way as to minimize wasted memory within a partition**

scanned_be_comp



(a) One process queue per partition

(b) Single process queue

Figure 7.3 Memory Assignment for Fixed Partitioning

sppu_te_be_comp

DYNAMIC PARTITIONING

- Partitions are of variable length and number
- Process is allocated exactly as much memory as required
- Eventually get holes in the memory. This is called **external fragmentation**
- Must use compaction to shift processes so they are contiguous and all free memory is in one block

sppu_te_be_comp

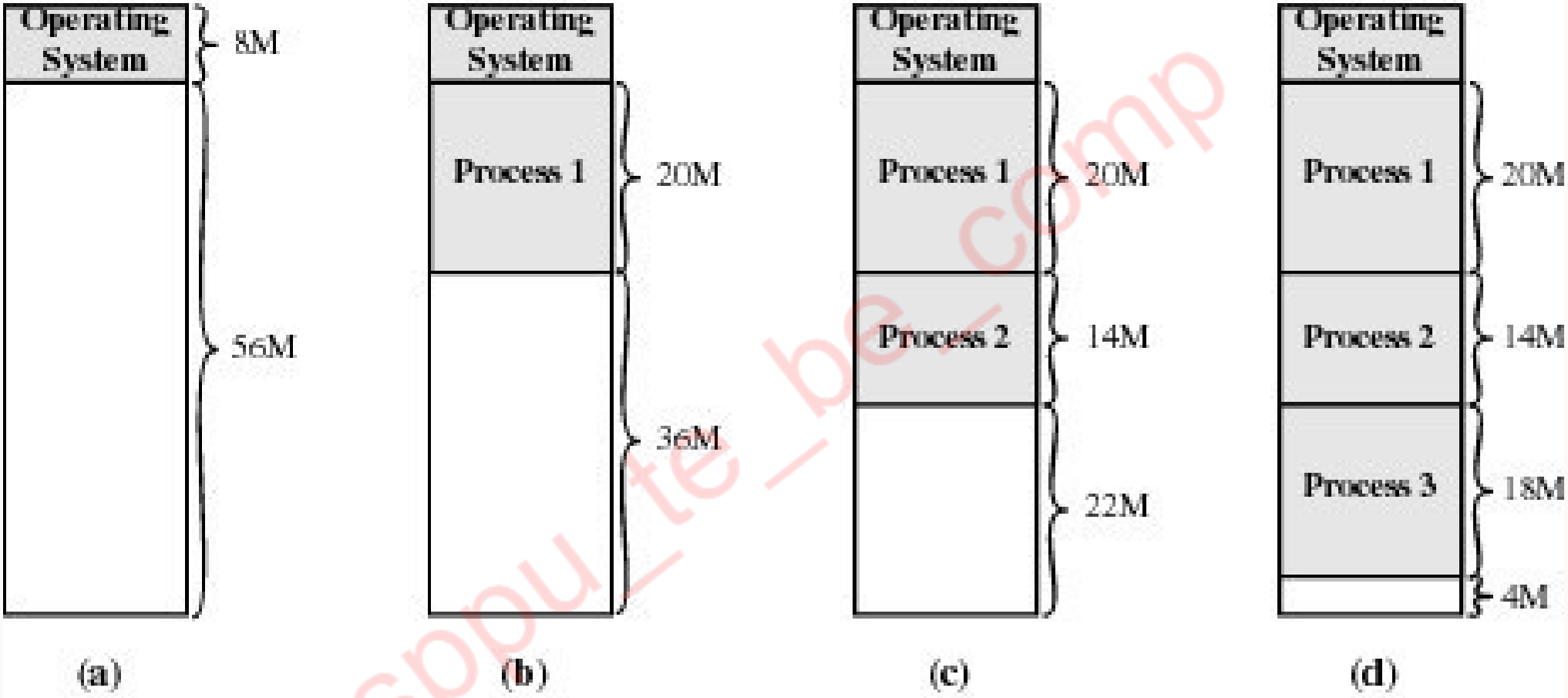


Figure 7.4 The Effect of Dynamic Partitioning

sppu_te_be_comp

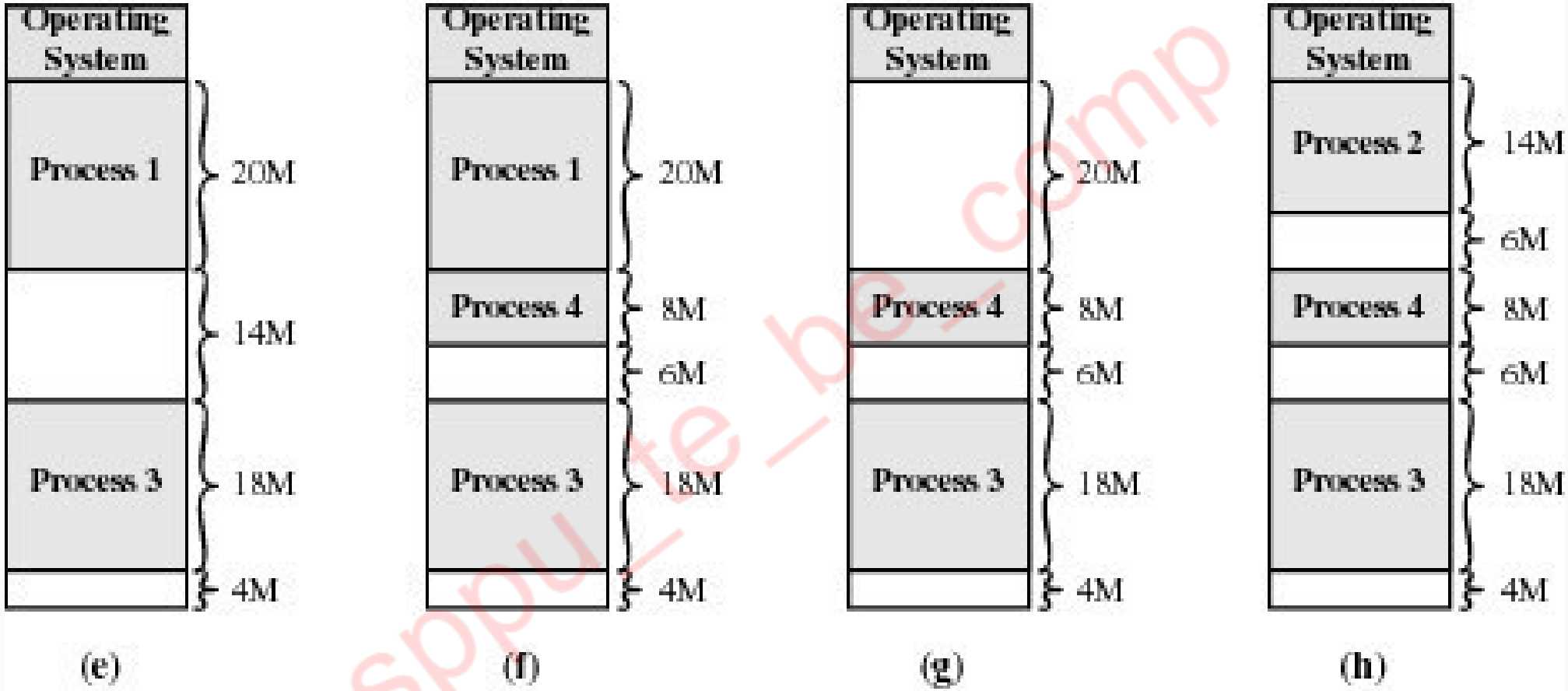


Figure 7.4 The Effect of Dynamic Partitioning

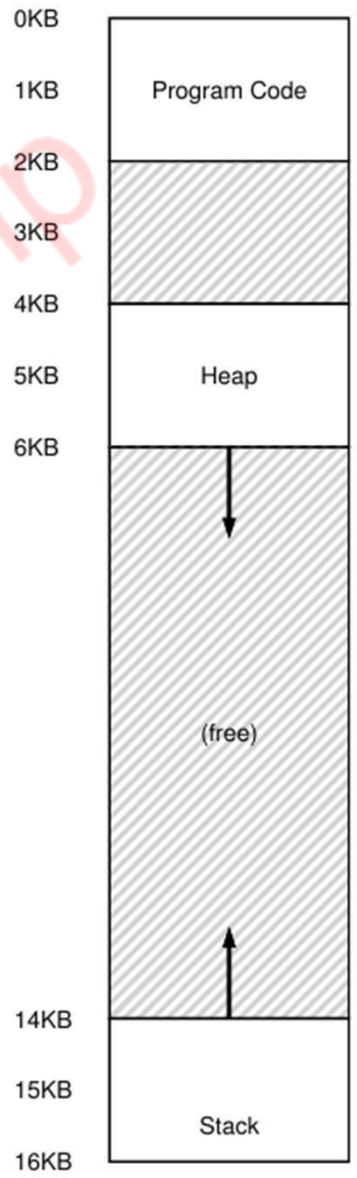
spu_te_be_comp

SEGMENTATION

- **Memory segmentation is a computer (primary) memory management method of division of a PC's primary memory into segments or sections.**
- **Segmentation lets the OS to divide a program's memory into logical sections (called segments) like:**
  - **Code (instructions)**
  - **Data (variables)**
  - **Stack (function calls and local variables)**
  - **Heap (dynamic memory)**

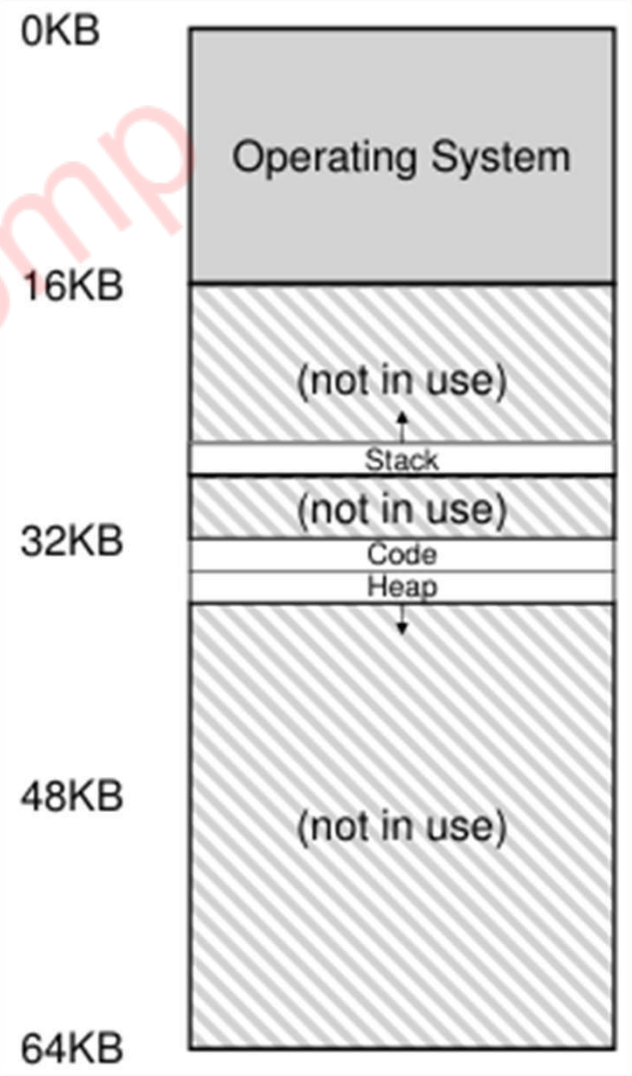
soon_to_be_comp

- **Consider the address space of a process which we want to place into physical memory:**



sppu_te_be_comp

- With a base and bounds pair per segment, we can place each segment independently in physical memory. As shown in the figure below:
- There you see a 64-KB physical memory with those three segments within it (and 16KB reserved for the OS).



SEGMENTATION CONTD…

- Each segment has a **name** and a **length.**
- The addresses specify both the segment name and the offset within the segment.
- For simplicity of implementation, segments are numbered and are referred to by a segment number, rather than by a segment name.
- Thus, a logical address consists of a two tuple:

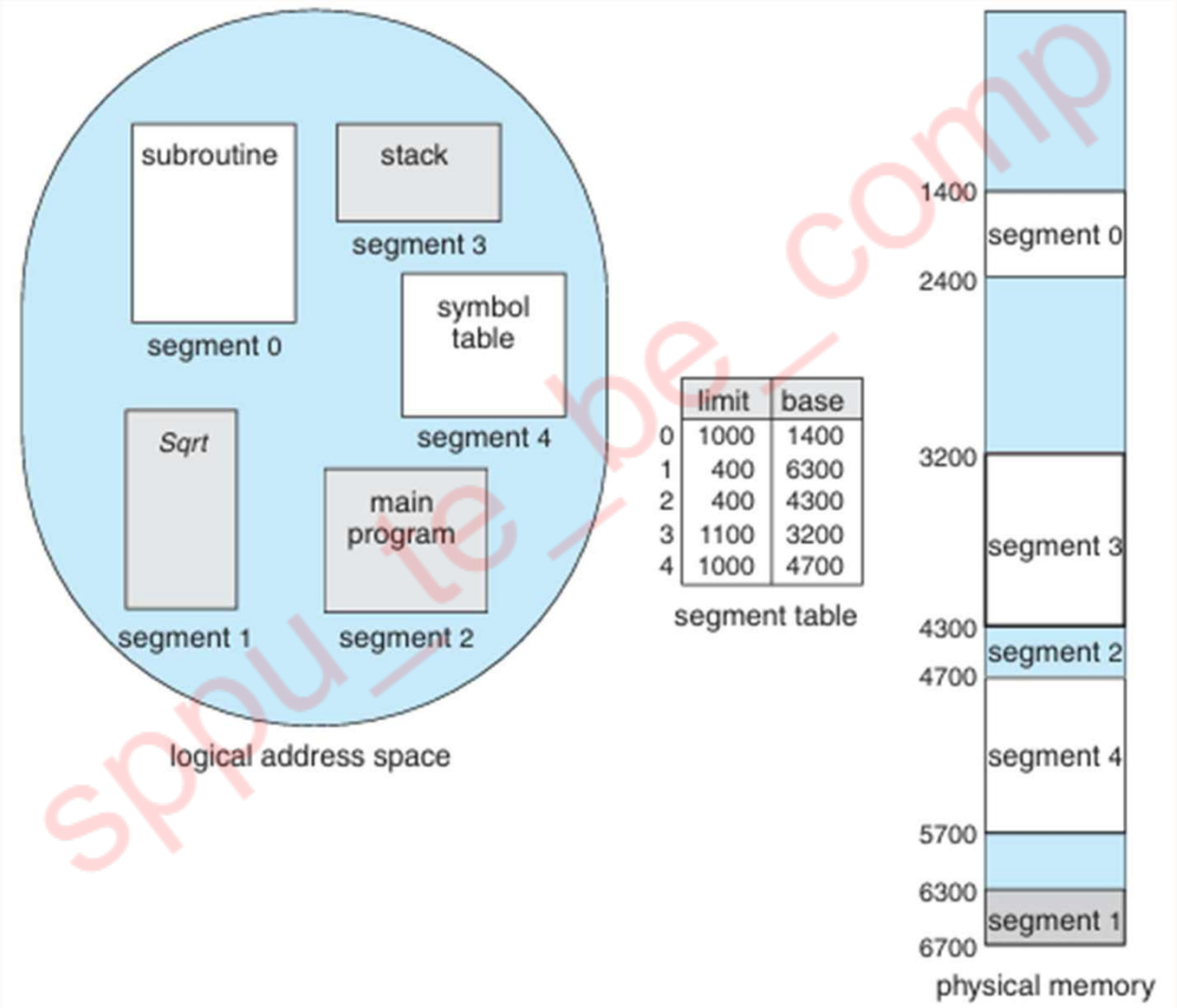
<segment-number, offset>

spnu_te_be_comp

- **How segmentation works:**
  - The operating system assigns segments to a program when it's loaded into memory.
  - Each segment has a base address (where it starts in memory) and a limit (its size).
  - The OS keeps track of these segments in a segment table.
  - When a program accesses a memory location, the logical address is translated into a physical address by the hardware, using the segment table.

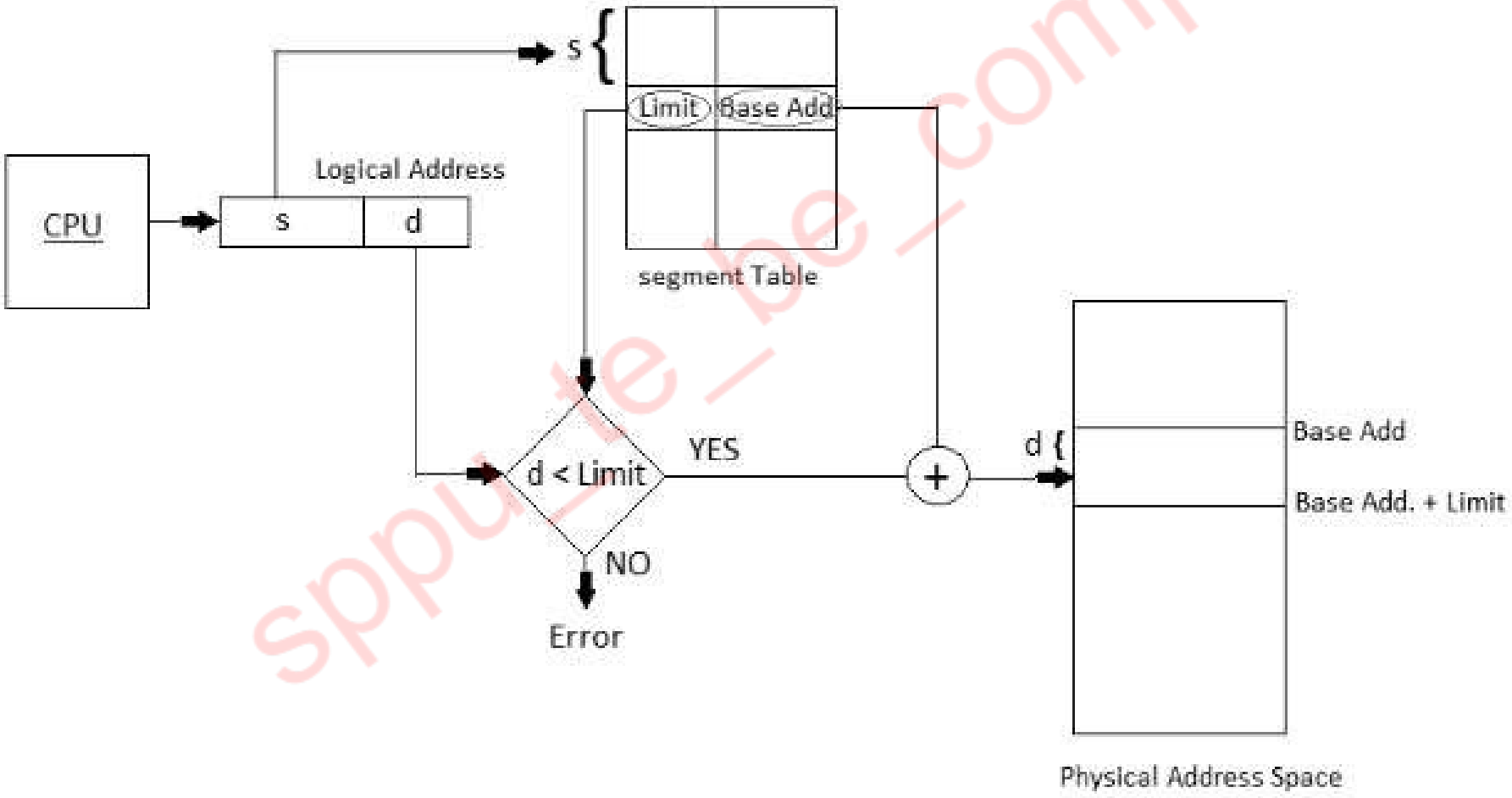
spu_te.he.comp

SEGMENTATION EXAMPLE



sppu_te_be_comp

SEGMENTATION HARDWARE



sppu_te_be_comp

- For example,

1. Segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location 4300 +53 = 4353.
2. A reference to segment 3, byte 852, is mapped to 3200 (the base of segment 3) + 852 = 4052.
3. A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1,000 bytes long.

spu_te_bc_comp

SEGMENTATION EXAMPLE

- **Consider the following segment table-**

Segment No.	Base	Length
0	1215	700
1	2300	14
2	50	100
3	1327	580
4	1552	5G

- **Calculate the physical address if no trap is produced for the following logical address.**

i. 0, 430

ii. 1, 11

iii. 2, 100

iv. 3, 425

v. 4, 55

spit_te_be_comp

FINE-GRAINED VS. COARSE-GRAINED SEGMENTATION

- **Coarse-Grained Segmentation:**
  - Involves dividing memory into a small number of large, high-level logical blocks or components, such as entire tables, monolithic applications, or large classes.
- **Fine-Grained Segmentation:**
  - Breaks memory into a large number of smaller, more detailed segments. This allows the operating system to manage and protect the memory in greater detail.

Should be comp

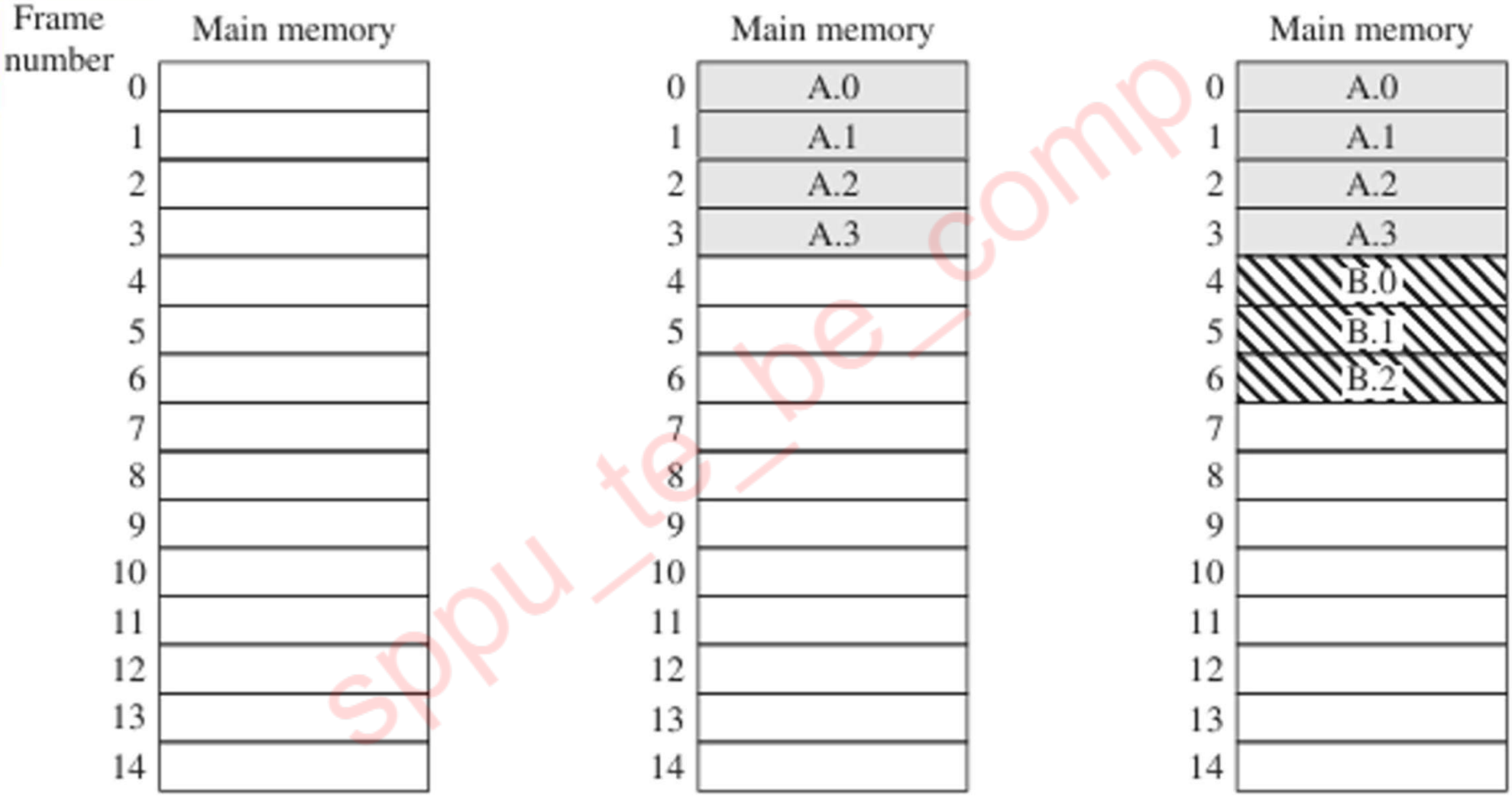
- **Key Differences between Coarse-Grained and Fine-Grained Segmentation:**
  - **Level of Detail**
  - **Complexity**
  - **Overhead**
  - **Application**

sppu_te_be_comp

PAGING

- **The basic method for implementing paging involves breaking**
  - Physical memory into fixed-sized blocks called **frames** and
  - Process into chunks is called **pages.**
- **Figure below shows the use of pages and frames.**
  - Process A, stored on disk, consists of four pages.
  - Process B, consists of three pages.
  - Process C consist of four pages and so on.

sppu_te_be_comp

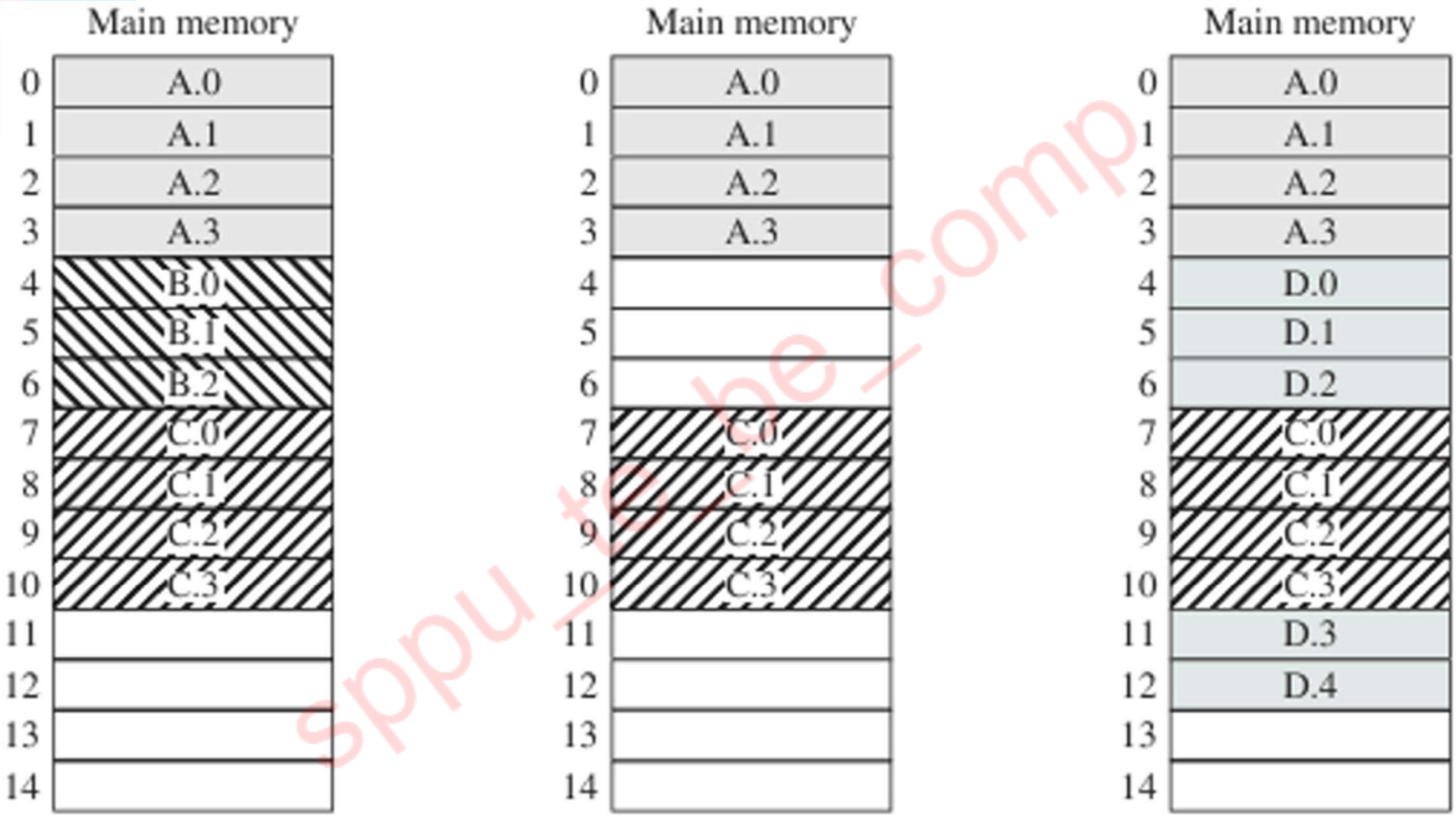


(a) Fifteen available frames

(b) Load process A

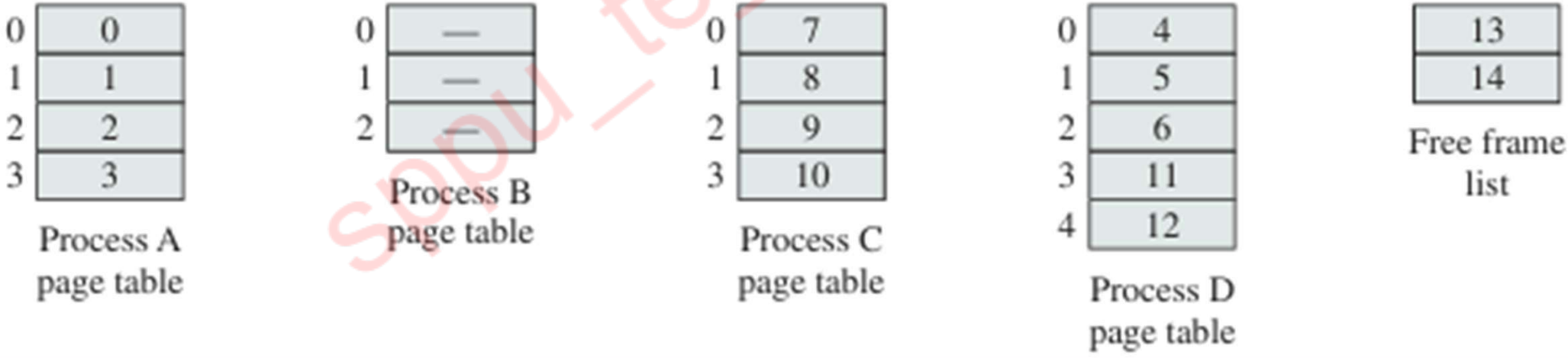
(c) Load process B

sppu_te_be_comp


Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4 B.0
5 B.1
6 B.2
7 C.0
8 C.1
9 C.2
10 C.3
11
12
13
14
(d) Load process C
Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4
5
6
7 C.0
8 C.1
9 C.2
10 C.3
11
12
13
14
(e) Swap out B
Main memory
0 A.0
1 A.1
2 A.2
3 A.3
4 D.0
5 D.1
6 D.2
7 C.0
8 C.1
9 C.2
10 C.3
11 D.3
12 D.4
13
14
(f) Load process D

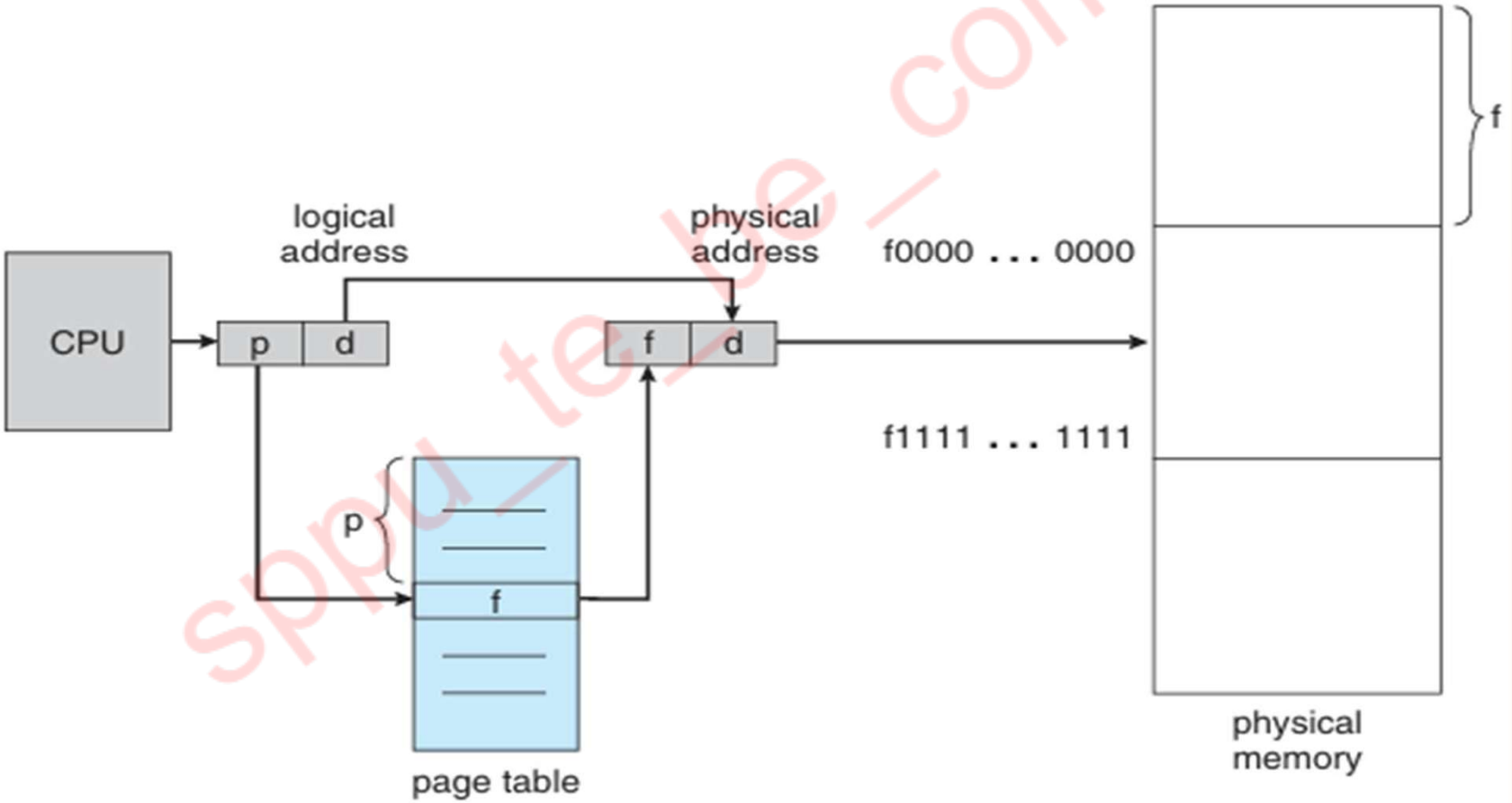
PAGING CONTD….

- **In case there is not sufficient contiguous frames to hold the process, then the pages will be loaded in any of the available frame.**
- **For this, OS maintains a page table for each process.**



sppu_te_be_comp

- **The hardware support for paging is illustrated in figure below:**



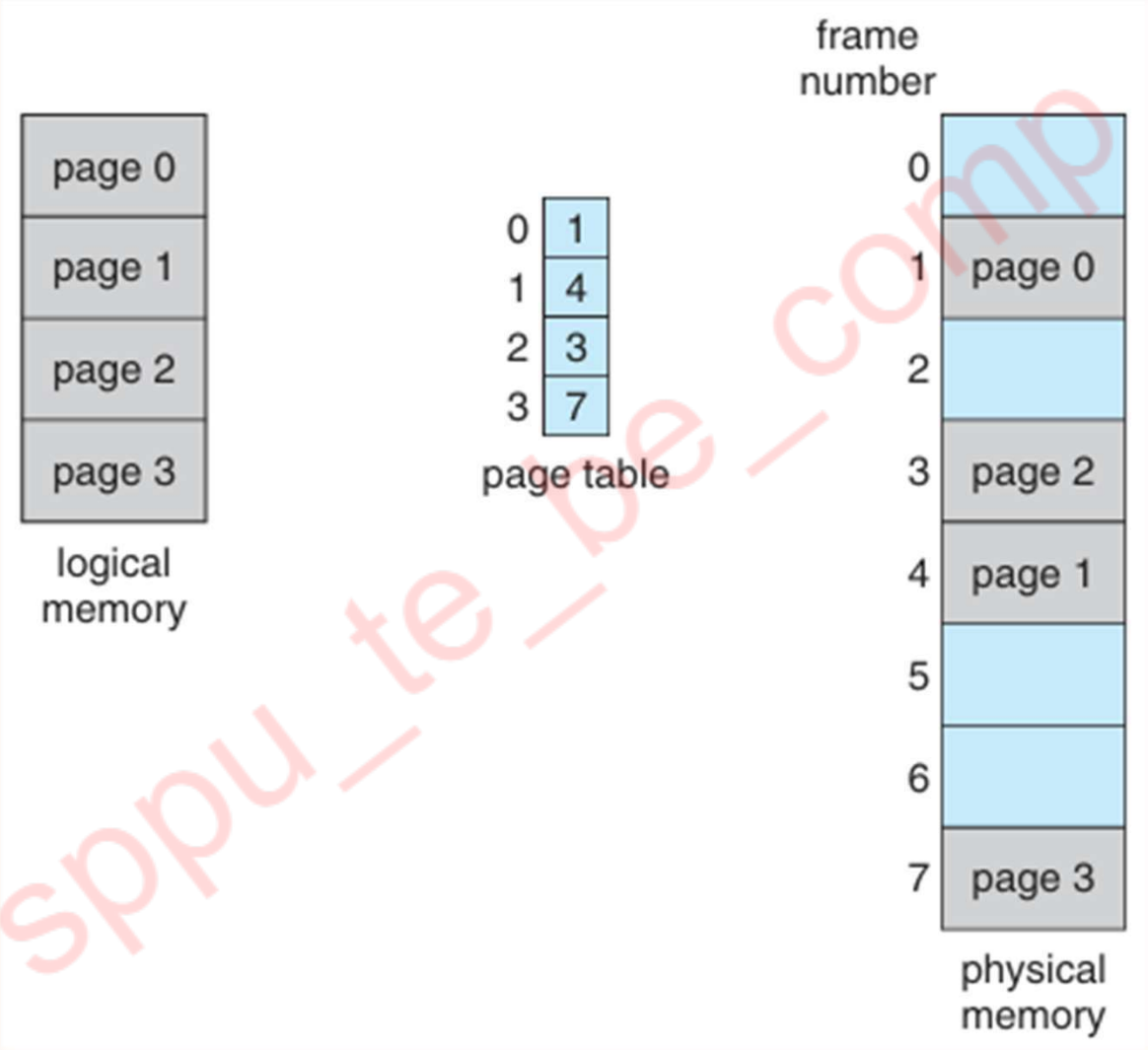
sppu_te_be_comp

- Every address generated by the CPU is divided into two parts: a page number(p) and a page offset(d).



- The page number is used as an index into a page table. The page table contains the base address of each page in physical memory.

sppu_te_he_comp



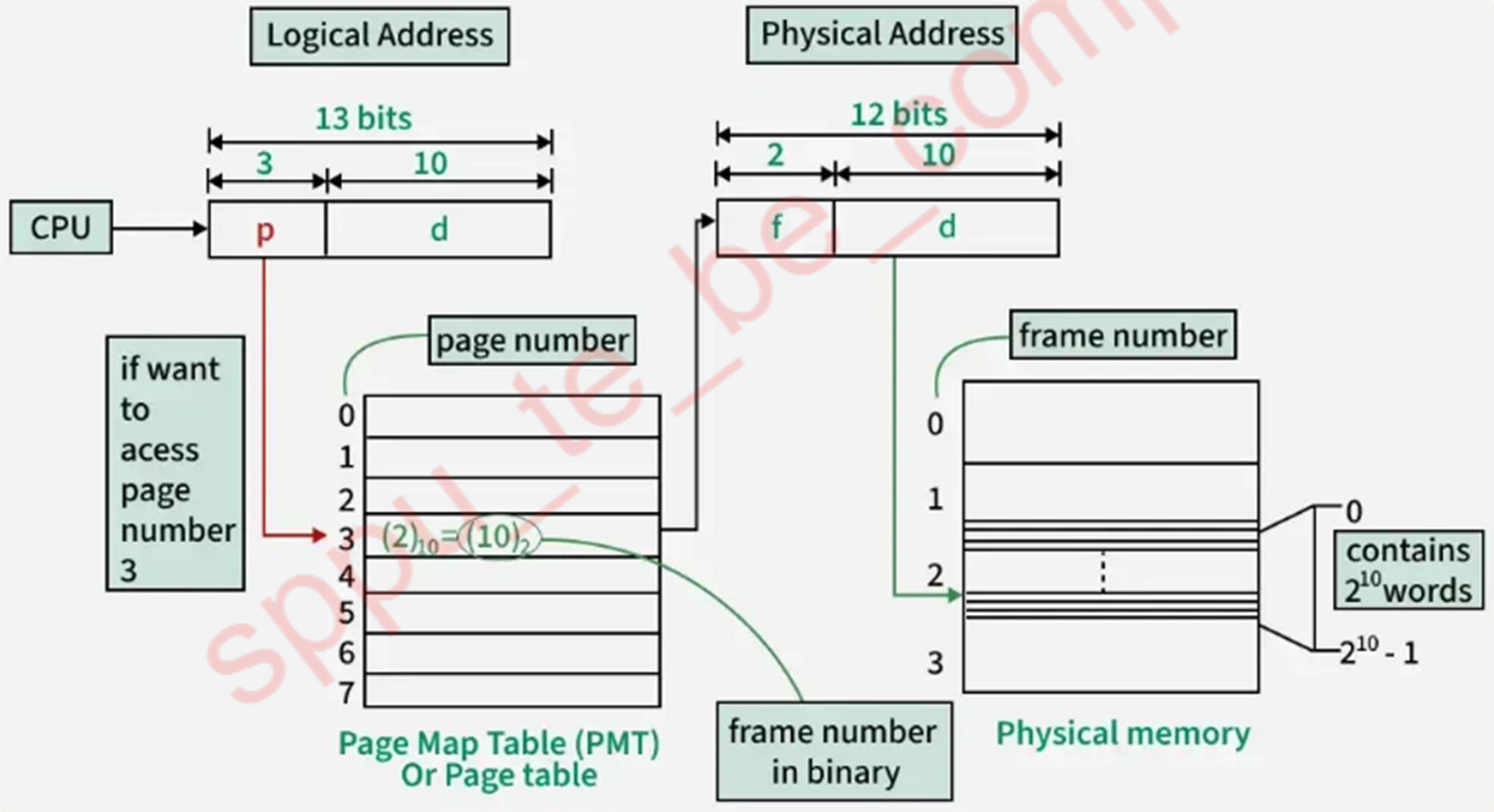
sppu_te_be_comp

IMPORTANT FEATURES OF PAGING

- **Logical to physical address mapping**
- **Fixed page and frame size**
- **Page table entries**
- **Number of page table entries**
- **Page table stored in main memory**

sppu_fe_be_comp

WORKING OF PAGING

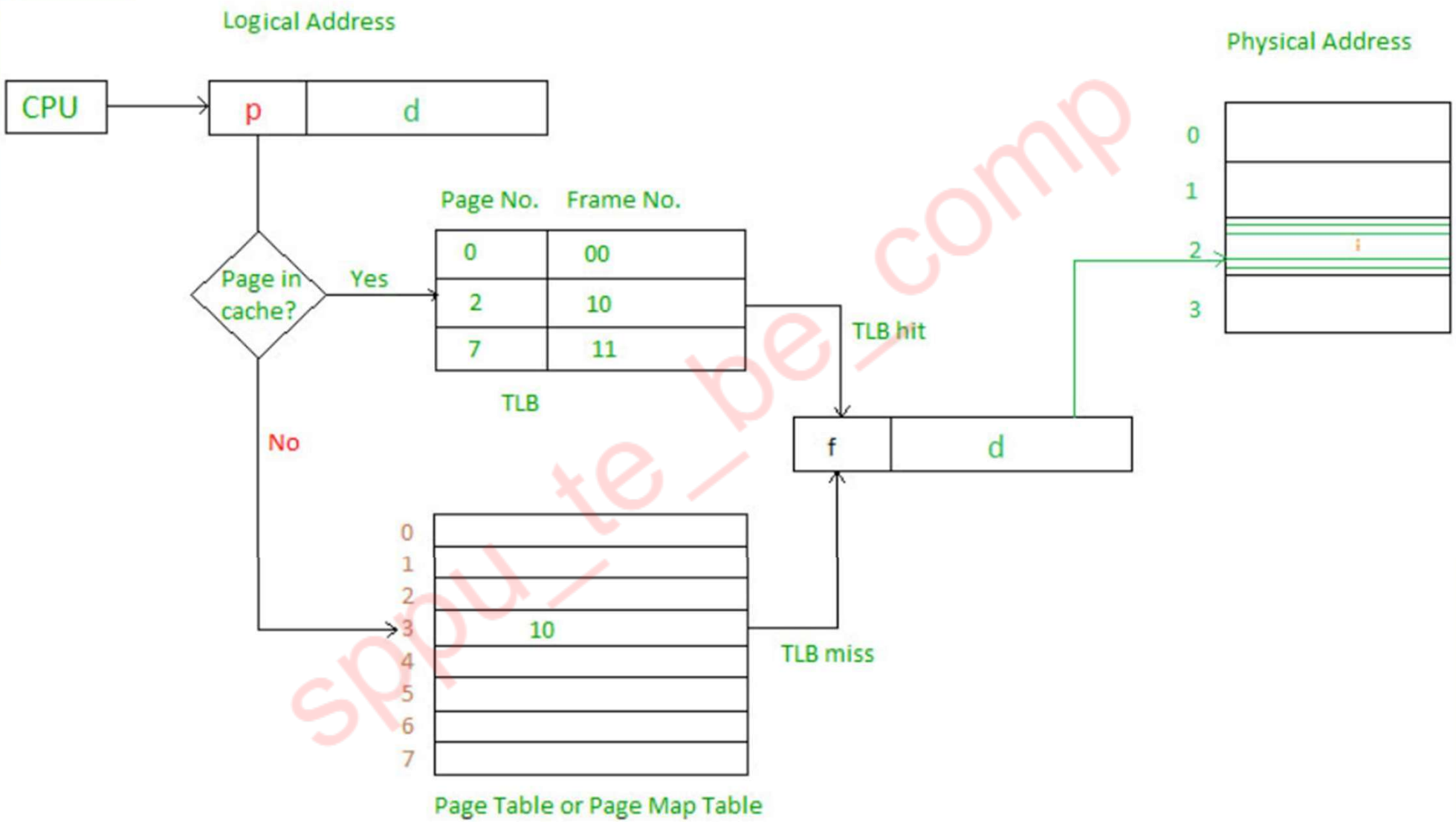


spu_te_be_comp

TLB

- Translation Lookaside Buffer, is a specialized cache memory within a computer's memory-management unit (MMU) that stores recently used virtual-to-physical address translations to speed up memory access.
- When the CPU needs to translate a virtual address to a physical one, it first checks the TLB.
  - A TLB hit means the translation is in the cache, allowing for a quick, direct access to the physical memory.
  - A TLB miss requires the CPU to access the full page table in main memory to find the translation, which is a slower process.

sppu_te_be_comp


Logical Address
Physical Address
CPU
p
d
Page No.
Frame No.
Page in cache?
Yes
0
00
2
10
7
11
TLB
TLB hit
No
f
d
0
1
2
3
10
4
5
6
7
TLB miss
Page Table or Page Map Table
0
1
2
3
i

- **Steps in TLB hit**
  1. **CPU generates a virtual (logical) address.**
  2. **It is checked in TLB (present).**
  3. **The corresponding frame number is retrieved, which now tells where the main memory page lies.**

sppu_tc_be_comp

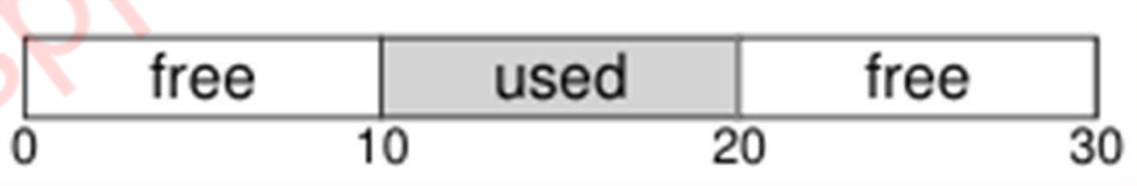
Steps in TLB miss

1. CPU generates a virtual (logical) address.
2. It is checked in TLB (not present).
3. Now the page number is matched to the page table residing in the main memory (assuming the page table contains all PTE).
4. The corresponding frame number is retrieved, which now tells where the main memory page lies.
5. The TLB is updated with new PTE (if space is not there, one of the replacement techniques comes into the picture i.e either FIFO, LRU or MFU etc).

spau_te_be_comp

FREE SPACE MANAGEMENT

- A system software in an OS that manipulates and keeps a track of free spaces to allocate and de-allocate memory blocks to files.
- When a file is created, the OS searches the free space list for the required space allocated to save a file.
- While deletion a file, the file system frees the given space and adds this to the free space list.



sppu_te_be_comp

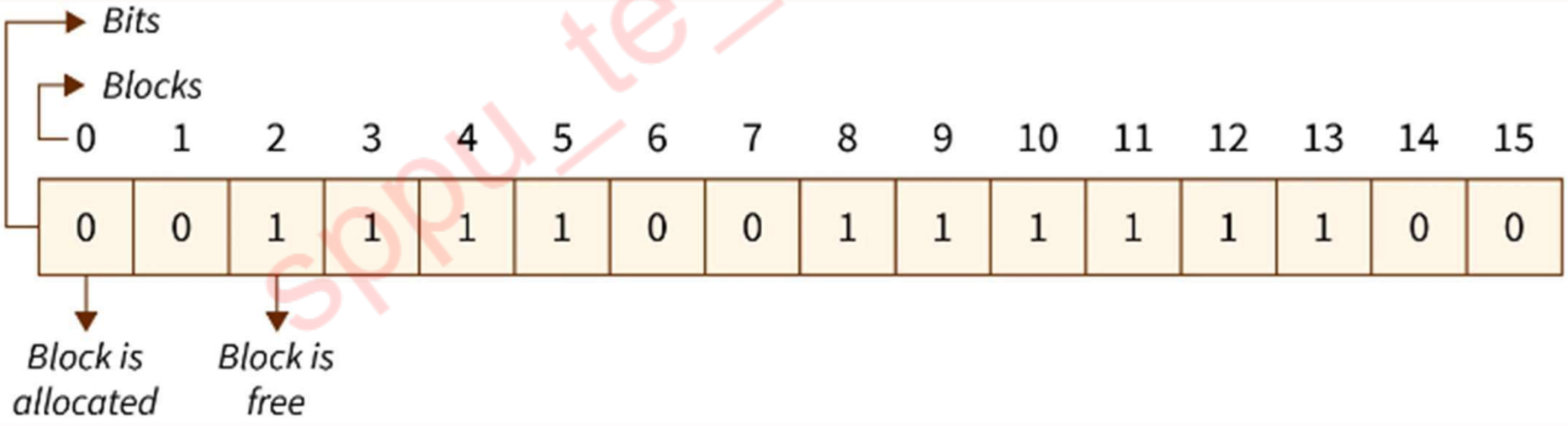
METHODS OF FREE SPACE MANAGEMENT IN OS

1. **Bitmap or Bit vector**
2. **Linked List**
3. **Grouping**
4. **Counting**

sppu_te_be_comp

1. BITMAP OR BIT VECTOR

- **It is a series or collection of bits in which each bit represents a disk block.**
  - If the block bit is 1, it means the block is empty and
  - If the block bit is 0, it means the block is not free.



sppu_te_be_comp

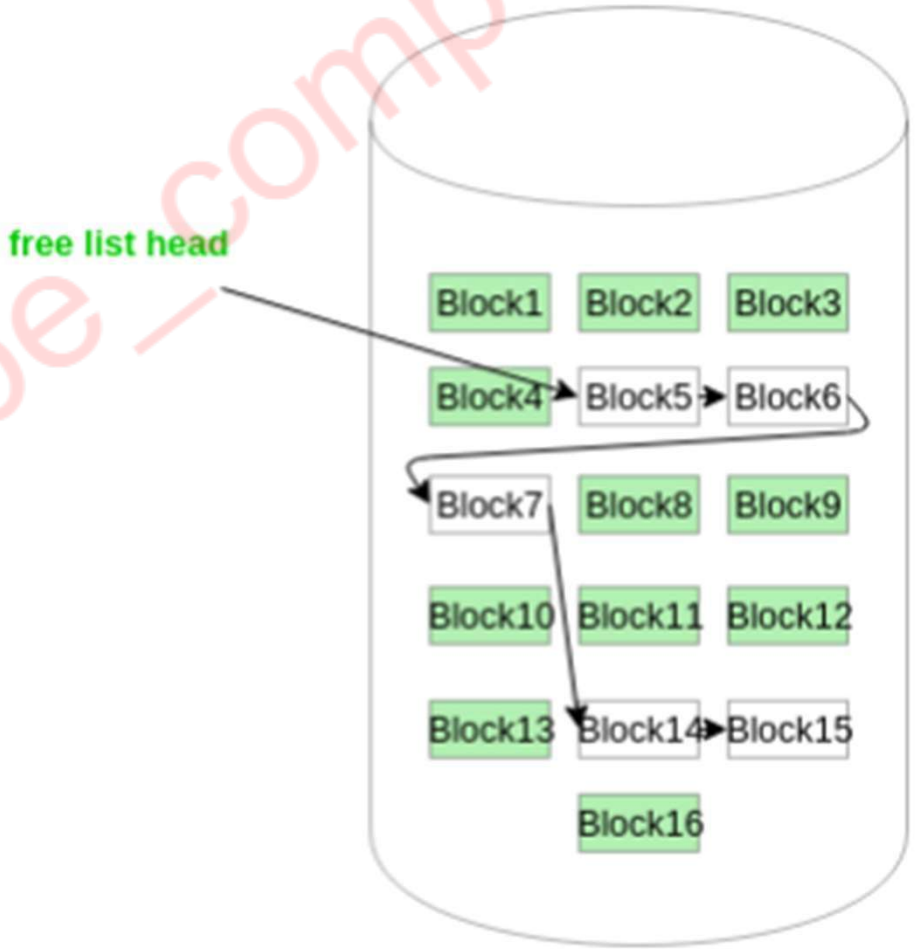
- **Advantages of Bit vector method**
  - Simple and easy to understand.
  - Consumes less memory.
  - It is efficient to find free space.

- **Disadvantages of the Bit vector method**
  - The operating system goes through all the blocks until it finds a free block. (block whose bit is '0').
  - It is not efficient when the disk size is large.

scpu_to_be_comp

2. LINKED LIST

- **Here all the free blocks inside a disk are linked together in a linked list. These free blocks on the disk are linked together by a pointer.**



sppu_fe_be_comp

- **Advantages:**
  - The total available space is used efficiently using this method.
  - Dynamic allocation in Linked List is easy, thus can add the space as per the requirement dynamically.

- **Disadvantages:**
  - When the size of Linked List increases, the headache of miniating pointers is also increases.
  - This method is not efficient during iteration of each block of memory.

sppu_te_be_comp

3. GROUPING

- **This approach stores the address of the free blocks in the first free block. The first free block stores the address of some, say n free blocks.**
- **Out of these n blocks, the first n-1 blocks are actually free and the last block contains the address of next free n blocks.**

Block 3 -> 4, 5, 6
Block 6 -> 9, 10, 11
Block 11 -> 12, 13, 14

sppu_fe_be_comp

- **Advantage:**
  - Finding free blocks in massive amount can be done easily using this method.

- **Disadvantage:**
  - The only disadvantage is, we need to alter the entire list, if any of the block of the list is occupied.

spit_te_be_comp

4. COUNTING

- **This approach stores the address of the first free disk block and a number n of free contiguous disk blocks that follow the first block.**
- **Every entry in the list would contain:**
  - Address of first free disk block.
  - A number n.

Block 3 -> 4
Block 9 -> 6

sppu_te_be_comp

- **Advantages:**
  - Using this method, a group of entire free blocks can take place easily and Fastly.
  - The list formed in this method is especially smaller in size.

- **Disadvantage:**
  - The first free block in this method, keeps account of other free blocks. Thus, due to that one block the space requirement is more.

scanu_te_be_comp

- **Advantages of Free Space Management Techniques**
  - Efficient Use of Storage Space
  - Easy to Implement
  - Faster Access to Files

- **Disadvantages of Free Space Management Techniques**
  - Fragmentation
  - Overhead
  - Limited Scalability
  - Risk of data loss

sppu_to_be_comp

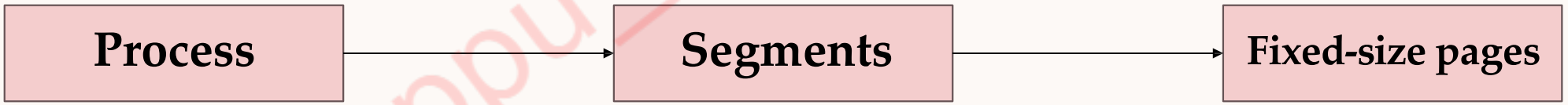
REPLACEMENT POLICY

- **Consider a reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. The number of frames in the memory is 3. Find out the number of page faults respective to:**
  - **FIFO Page Replacement Algorithm**
  - **LRU Page Replacement Algorithm**
  - **MRU Page Replacement Algorithm**
  - **Optimal Page Replacement Algorithm**

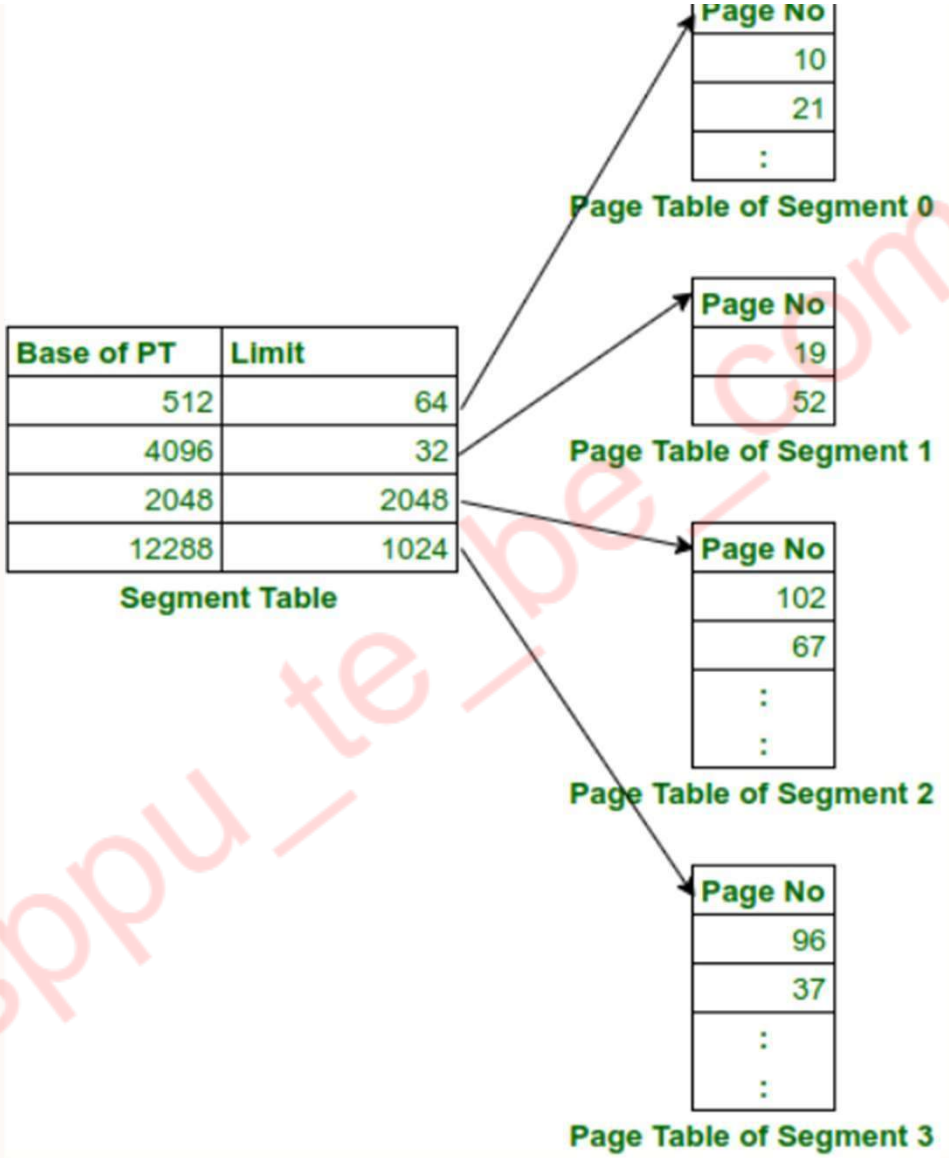
sppu_te_be_comp

HYBRID APPROACH: PAGING AND SEGMENTATION

- **A hybrid approach to memory management combines the logical, flexible division of segmentation with the efficient, fixed-size blocks of paging, creating "segmentation with paging" or "segmented paging".**

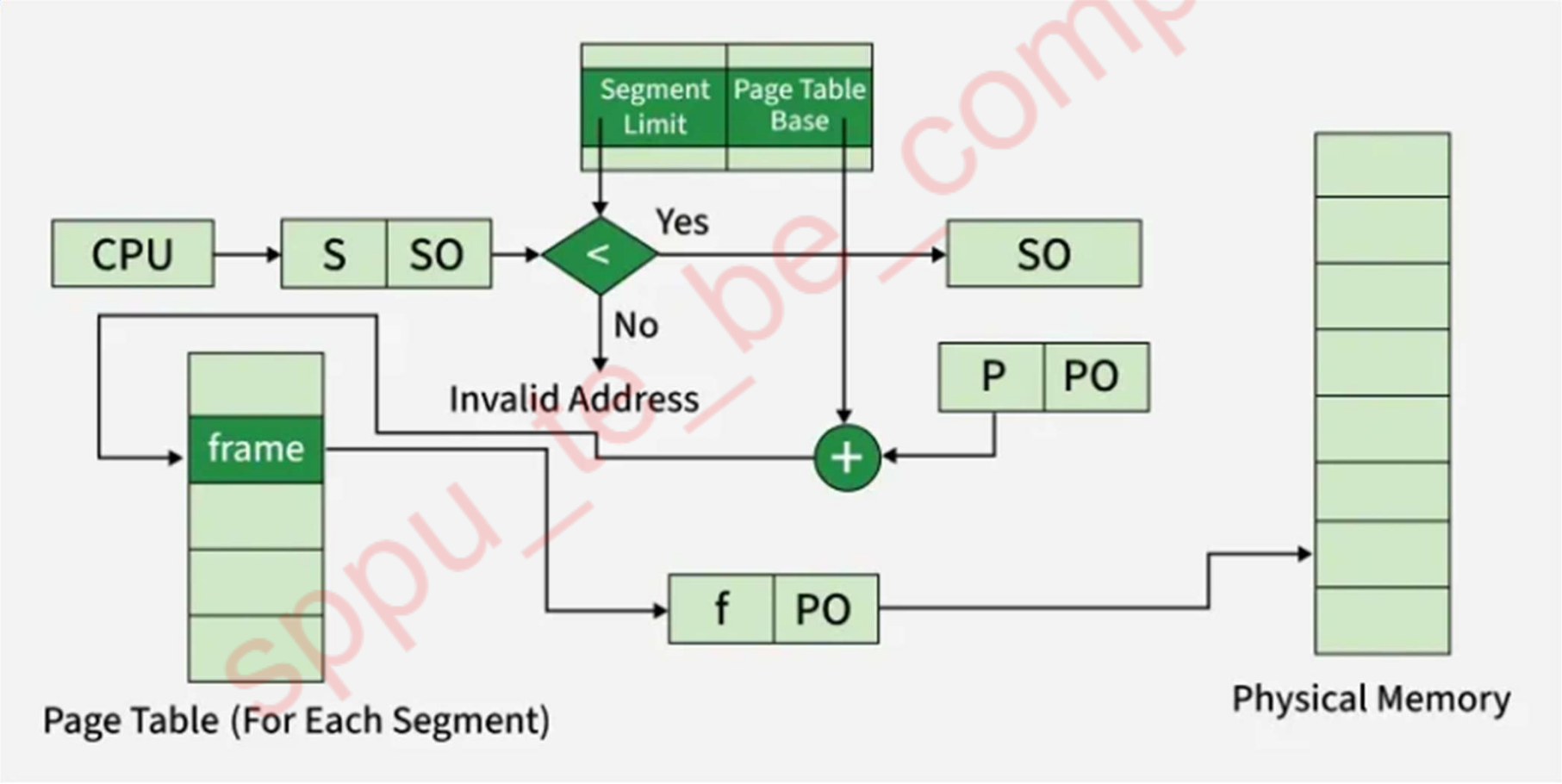


sppu_te_be_comp



sppu_te_be_comp

- **Below is a complete workflow diagram of Segmented Paging:**



Workflow of Segmented Paging

sppu_to_be_comp

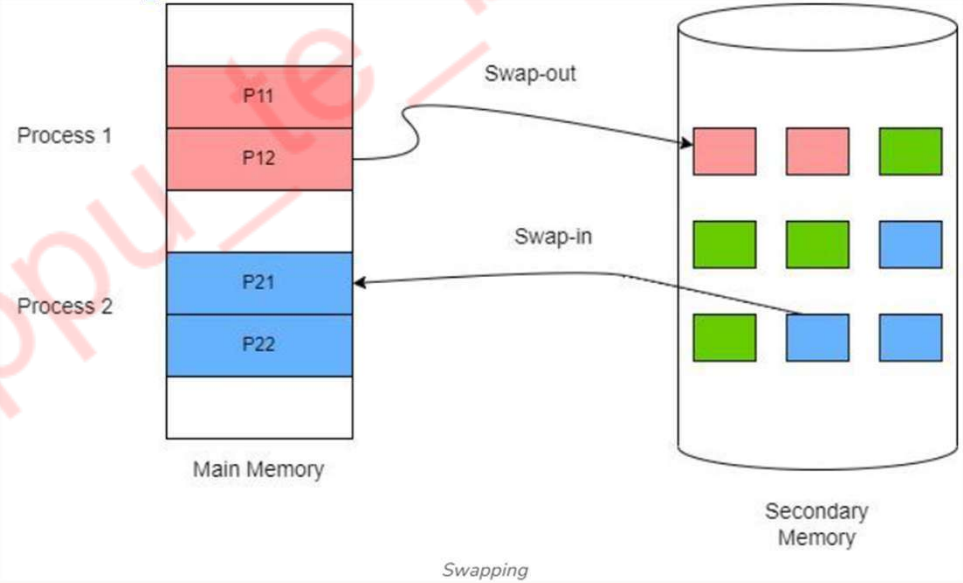
BEYOND PHYSICAL MEMORY: MECHANISMS

- **Beyond just a single process, the addition of swap space allows the OS to support the illusion of a large virtual memory for multiple concurrently running processes.**
- **The invention of multiprogramming almost demanded the ability to swap out some pages, as early machines clearly could not hold all the pages needed by all processes at once.**
- **Thus, the combination of multiprogramming and ease-of-use leads us to want to support using more memory than is physically available.**

sppu_te_be_comp

1. SWAP SPACE

- **Swap space enables the OS to handle memory more flexibly, supporting the illusion of a larger memory than physically available, thus improving multitasking and stability of the system.**



Swapping

sppu_te_be_comp

- **How swap space works:**

1. **Swap-out:** When the physical RAM is nearly full unused blocks of memory, called "pages," and moves them from RAM to the swap space.
2. **Swap-in:** If a process later needs access to data that has been moved to swap space, the OS swaps that data back into RAM.
3. **Virtual Memory:** The combination of RAM and swap space is referred to as virtual memory.

spu_to_be_comp

3. PAGE FAULT CONTROL FLOW

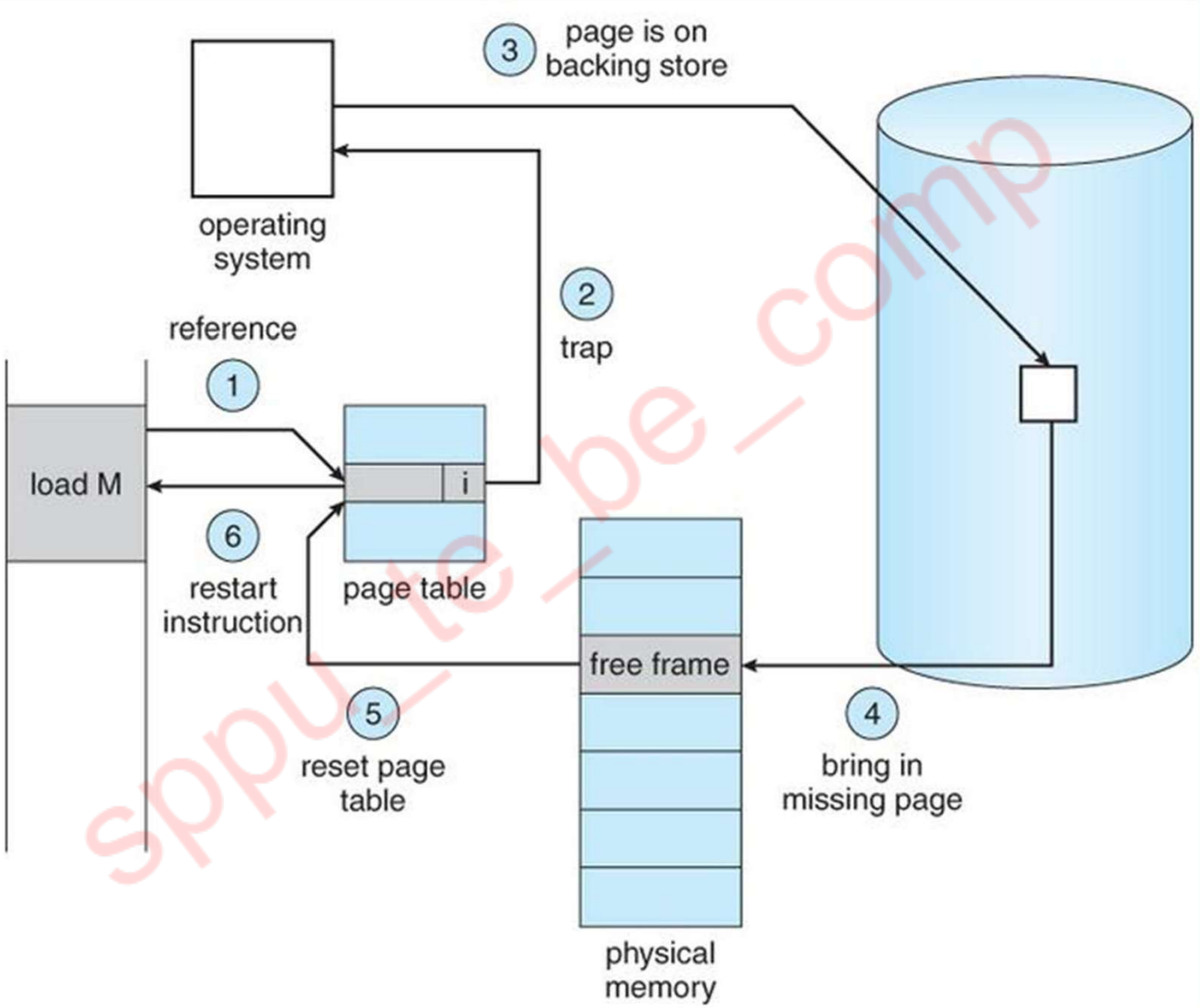
- **Page fault control flow is the sequence of steps an operating system takes when a program tries to access a memory page not currently in RAM.**
- **The process begins with hardware detecting the missing page, which generates a trap, transferring control to the OS's page fault handler.**

sppu_te_be_comp

- **The typical steps involved in the page fault control flow are defined as follows:**

1. Page Fault Detection (Trap to OS)
2. Save State
3. Identify the Page
4. Address Validation
5. Find space in RAM
G. Page Replacement (if necessary)
7. Load Page from Disk
8. Update Page Table
5. Resume Process

sppu_te_be_comp



sppu_te_be_comp

THANK YOU

sppu_te_be_comp